

GAL Compiler — Defense-Prep Walkthrough

File 3 of 9: `lexer.py` — the `Lexer` class itself

This is the **scanner** (lexical analyzer) of GAL. It is where raw character text becomes a stream of tokens. The file is large (~1900 lines) because GAL has 26 reserved words, 5 data types, ~20 operators, 5 literal kinds, two comment styles, and strict per-keyword delimiter rules. The entire file is built around **one big function**: `Lexer.make_tokens()`.

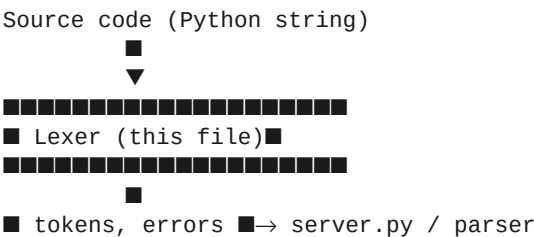


1. FILE PURPOSE

`lexer.py` performs **lexical analysis** — phase 1 of compilation. Its responsibilities:

1. Read the GAL source code character by character.
2. Group characters into **lexemes** and tag each with a **token type** (a `Token` object).
3. **Track line and column** so every token knows where it came from in the source.
4. **Detect lexical errors** (illegal characters, malformed numbers, unclosed strings/comments, bad delimiters) and produce `LexicalError` objects — without crashing.
5. **Skip whitespace and comments** entirely (they never become tokens).
6. **Recognize keywords vs identifiers** with a transition-diagram-style hand-written FSM, one branch per starting letter.
7. Emit a final `EOF` token so the parser knows when input ends.

Where it sits in the pipeline:



What depends on it:

File	What it imports	When used
<code>server.py</code>	<code>lex</code> , <code>get_token_description</code>	Every API endpoint runs <code>lex(source_code)</code> first
<code>Gal_Parser.py</code>	(consumes tokens, doesn't import the lexer module)	Receives tokens to drive the LL(1) parser

File	What it imports	When used
GALsemantic.py	Lexer (for some tests)	Uses <code>analyze_semantics(tokens)</code> legacy path
icg.py	(consumes tokens)	Reads the same token stream to emit display TAC

The lexer **does not** depend on anything other than the token/Position/error classes that live at the top of the same file. That makes it easy to test in isolation.



2. IMPORTS / DEPENDENCIES

The lexer file has zero `import` statements. Everything it needs — the character classes, the token-type constants, the `Position`, `LexicalError`, and `Token` classes — is defined in the same file (the part covered in defense file #2 of this series).

This is intentional. The lexer must be a pure data-processing function with no side effects beyond producing tokens. By avoiding imports, we guarantee:

- It can be unit-tested without setting up Flask, Socket.IO, or eventlet.
- It has no hidden global state from other modules.
- Removing or renaming any import in `server.py` cannot break tokenization.

Defense answer: *"The lexer is self-contained. The only data structures it uses are defined in the same file. This isolates the most-tested, most-critical phase of the compiler from runtime concerns."*



3. GLOBAL CONSTANTS / VARIABLES

The lexer relies on the constants defined at the top of `lexer.py` (these were covered in defense file #2). Briefly:

- **Character classes** — `ZERO`, `DIGIT`, `ZERODIGIT`, `LOW_ALPHA`, `UPPER_ALPHA`, `ALPHA`, `ALPHANUM` — used to test `if self.current_char in ALPHA:` etc.
- **Delimiter sets** — `space_delim`, `delim2 ... delim24`, `idf_delim`, `whlnum_delim`, `decim_delim` — each one is the set of characters that may legally follow a particular kind of token. The actively-used ones (`space_delim`, `delim4`, `delim6`, `delim7`, `delim8`, `delim23`, `idf_delim`, `whlnum_delim`, `decim_delim`) drive lookahead checks in the scanner. The numbered ones are kept as 1-to-1 documentation mapping back to the GAL proposal's regular-definition table.
- **Token-type constants** (`TT_*`) — what each emitted `Token` is tagged with.

The `Lexer` instance itself owns three pieces of state:

```

self.source_code # the raw input string
self.pos         # a Position(index, line, col) – current scan location
self.current_char # the character at self.pos.index, or None at EOF

```

That's it. No caches, no counters outside the loop. Everything else lives in local variables of `make_tokens()`.



4. CLASSES AND FUNCTIONS

This file exposes one class (`Lexer`) and two module-level functions (`run` and `lex`). The `Lexer` class has three methods.

4.1 `Lexer.__init__(source_code)` (lines 307-311)

```

def __init__(self, source_code):
    self.source_code = source_code.replace('\r', '')
    self.pos = Position(-1, 1, -1)
    self.current_char = None
    self.advance()

```

Receives: the full GAL source code as a Python string.

Returns / modifies: populates the three instance fields and primes `current_char` to the first character of the source.

Why it exists: sets up scanner state. The `\r` strip handles Windows line endings (`\r\n`) so the rest of the lexer only ever has to consider `\n`.

Why `Position(-1, 1, -1)`: the position starts *before* the first character. The very first call to `self.advance()` will move it to index 0, line 1, column 0 — the actual start of the source.

Compiler stage: Lexical (setup).

4.2 `Lexer.advance()` (lines 313-317)

```

def advance(self):
    self.pos.advance(self.current_char)
    self.current_char = (
        self.source_code[self.pos.index]
        if self.pos.index < len(self.source_code) else None
    )

```

Receives: nothing.

Returns / modifies: `self.pos` is incremented (line/column updated based on the *previous* `current_char`); `self.current_char` is set to the next source character or `None` if past the end.

Why it exists: every scanner branch in `make_tokens` calls `self.advance()` to move forward by one character. Centralizing it ensures consistent line/column tracking.

Edge case: when EOF is reached, `self.current_char` becomes `None`. Every scanner branch checks `is not None` before reading.

4.3 `Lexer.make_tokens()` (lines 319-1832 — the heart of the lexer)

```
def make_tokens(self):
    tokens = []
    line = 1
    errors = []
    pos = self.pos.copy()
    while self.current_char != None:
        # ... one big if-elif-else over current_char ...
        if self.current_char is None:
            tokens.append(Token(TT_EOF, "", line, pos.col))
    return tokens, errors
```

Receives: nothing (operates on `self.source_code`).

Returns: the tuple `(tokens, errors)` — a list of `Token` objects and a list of `LexicalError` objects.

Why it exists: this is the scanner. Every character of the source goes through this loop exactly once. Each iteration recognizes one lexeme and emits at most one token (whitespace and comments emit nothing).

Edge cases:

- Empty source: the `while` loop never runs, EOF is appended, `(([EOF], []))` is returned.
- Source with only comments: tokens list contains only EOF.
- Source with only errors: tokens list may still contain valid tokens before/after each error.

Defense framing: *"Make_tokens is one big dispatch. The first character of each lexeme decides which branch to take. Inside each branch we keep reading characters as long as they belong to the current token, then emit it. Errors are collected — never raised — so we can report multiple problems in one pass."*

4.4 `run(source_code)` and `lex(source_code)` module-level helpers (lines 1838-1880)

```
def run(source_code):
    """Legacy function - runs lexer and returns tokens and errors."""
    lexer = Lexer(source_code)
    return lexer.make_tokens()

def lex(source_code):
    """Main entry point for lexical analysis (used by server.py)."""
    lexer = Lexer(source_code)
    tokens, errors = lexer.make_tokens()
    str_errors = [e.as_string() if hasattr(e, 'as_string') else str(e)
                  for e in errors]
    return tokens, str_errors
```

Why two:

- `run()` is the **legacy** entry point (kept for older grading/test scripts that imported it). Returns raw `LexicalError` objects.

- `lex()` is the **production** entry point used by `server.py`. It additionally converts each error into its formatted string via `as_string()`, because the HTTP/JSON layer needs strings, not Python objects.

Stage: Lexical (public interface).

Defense answer for "why two?": *"run is kept for backwards compatibility with grading scripts. lex is what the server uses — it adds error-to-string conversion so the JSON response is ready to serialize."*



5. LINE-BY-LINE / BLOCK-BY-BLOCK EXPLANATION

The body of `make_tokens` is a huge `if ... elif ... elif ... else` chain. I'll walk the most important branches in the order they appear. Every branch follows the same pattern: detect the start character → consume the lexeme → emit a token (or an error) → `continue` the loop.

5.1 The keyword/identifier scanner — letter-by-letter FSM (lines 338-940)

```
if self.current_char in ALPHA:
    ident_str = ''
    pos = self.pos.copy()

    if self.current_char == "b":
        ident_str += self.current_char
        self.advance()
        if self.current_char == "r": # branch
            ident_str += self.current_char
            self.advance()
            if self.current_char == "a":
                ...
                # eventually:
                if self.current_char is None or self.current_char in space_delim:
                    tokens.append(Token(TT_RW_BRANCH, ident_str, line, pos.col))
                    continue
            elif self.current_char == "u":
                ... # bud / bundle
        elif self.current_char == "c":
            ... # cultivate
        elif self.current_char == "e":
            ... # empty
        elif self.current_char == "f":
            ... # frost / fertile
        elif self.current_char == "g":
            ... # grow
    # ... and so on for h, l, p, r, s, t, v, w
```

What this block does: Hand-written transition diagram. For each letter the source could start with that begins a GAL keyword, there is an explicit nested `if` chain that walks character-by-character down the keyword's spelling. If the chain completes AND the next character is a valid delimiter (space, `;`, `{`, etc.), a keyword `Token` is emitted.

Why hand-written and not regex/dictionary?

- **Visibility for the panel:** this is the implementation of the **transition diagrams** in the GAL compiler proposal. Each `if` chain corresponds to one DFA path in the spec. A panelist can point at any `if "h"` branch and it maps directly to the proposal's "harvest" transition diagram.
- **Per-keyword delimiter rules:** GAL is strict about what may come after each keyword. `bud` requires `:` or `(` to follow (`delim4`); `fertile` requires `;` (`delim8`); `plant` allows `;`, `(`, `,`, etc. (`delim6`). With a regex-based approach, this would all be a separate validation step. With the hand-written FSM, the delimiter check is in-line right where the keyword is recognized.
- **Identifier fallback:** if the chain breaks (e.g., `b` is followed by `e`, not `r` or `u`), control falls through to a generic "scan an identifier" loop that runs at the end of the block (lines 802-940 in the file). The result is `Token('id', ident_str, ...)`.

Why the spec-derived scanner uses delimiter sets:

```
if self.current_char is None or (self.current_char is not None and self.current_char.isspace()) or self.current_char in self.delim4:
    tokens.append(Token(TT_RW_CULTIVATE, ident_str, line, pos.col))
    continue
elif self.current_char is not None and not self.current_char.isspace() and self.current_char not in self.delim4:
    errors.append(LexicalError(pos, f"Invalid delimiter '{self.current_char}' after '{ident_str}'"))
    self.advance()
    continue
```

This pattern repeats for every keyword. The two cases are:

1. **Valid delimiter** (`delim4` for `cultivate`) — emit the keyword token, continue.
2. **Invalid non-alphanumeric delimiter** (e.g., `cultivate$`) — report a delimiter error.

The third implicit case — character is alphanumeric — means the keyword spelling matched a *prefix* but the user actually typed a longer identifier (e.g., `cultivate1`). Control falls out of the keyword branch and into the generic identifier loop, which builds `Token('id', 'cultivate1')`.

Defense answer: *"The keyword scanner is a hand-written transition diagram, one branch per starting letter. This mirrors the FSMs documented in our compiler proposal. After spelling out each keyword, we check that the next character is a valid delimiter for that specific keyword — for example, `cultivate` may only be followed by `:`, `(`, or whitespace because it's a control-flow header. If the next character is alphanumeric instead, we treat the prefix as part of an identifier."*

5.2 The negative-literal scanner — ~ (lines 991-1049)

This is the most distinctive part of GAL — `~` is the negative sign.

```
elif self.current_char == "~": # Added for negative prefix
    ident_str = self.current_char
    pos = self.pos.copy()
    self.advance()

    # If ~ is directly followed by a digit, consume the number as a negative literal
    if self.current_char is not None and self.current_char in ZERODIGIT:
        # Read integer digits
        num_str = ""
        integer_digit_count = 0
        while self.current_char is not None and self.current_char in ZERODIGIT:
            integer_digit_count += 1
            num_str += self.current_char
            self.advance()
```

```

# Check for decimal point (negative double)
if self.current_char == ".":
    ...
    tokens.append(Token(TT_DOUBLELIT, ident_str, line, pos.col))
    continue
else:
    # Negative integer
    if integer_digit_count > 8:
        errors.append(LexicalError(pos, f"Integer exceeds maximum of 8 digits"))
        continue
    num_str = num_str.lstrip("0") or "0"
    ident_str = "~" + num_str
    tokens.append(Token(TT_INTEGERLIT, ident_str, line, pos.col))
    continue

# ~ not followed by a digit: emit as negate operator
elif self.current_char is None or self.current_char in ALPHANUM + ' \t\n':
    tokens.append(Token(TT_NEGATIVE, ident_str, line, pos.col))
    continue
else:
    errors.append(LexicalError(pos, f"Invalid delimiter '{self.current_char}' after '{ident_str}'."))
    self.advance()
    continue

```

What this block does: The lexer treats `~` two different ways depending on what comes next:

1. `~` **followed by a digit** (e.g., `~5`, `~3.14`) — fold the sign into a literal token. The result is a single `Token('intlitt', '~5', ...)` or `Token('dbllitt', '~3.14', ...)`. The `~` stays in the value string so the interpreter can recognize it later and emit a real negative number.
2. `~` **followed by an identifier or a paren** (e.g., `~x`, `~(a + b)`) — emit a separate `Token('~', '~', ...)` (the `TT_NEGATIVE` operator). The parser's grammar treats this as a unary minus prefix on an expression.

Why both behaviors: for literals, folding the sign at lex time means the parser never needs a "unary minus" production in front of every integer or double — the literal already carries the sign. For variables, we *can't* fold because the value isn't known yet, so we keep the operator as a separate token.

Defense answer: "GAL uses `~` for negation. The lexer pre-folds `~5` into a single negative integer literal at scan time, but emits `~` as a separate operator token when it precedes an identifier. This avoids a unary-minus production in the grammar for the literal case."

5.3 Maximal-munch operator scanners (lines 1051-1283)

```

elif self.current_char == "!":
    ident_str = self.current_char
    pos = self.pos.copy()
    self.advance()
    if self.current_char == "=":
        ident_str += self.current_char
        self.advance()
        tokens.append(Token(TT_NOTEQ, ident_str, line, pos.col))
        continue
    else:
        tokens.append(Token(TT_NOT, ident_str, line, pos.col))
        continue

```

Pattern: when the current character could start either a 1-char or a 2-char operator (e.g., `!` could be NOT or `!=`), we read the first char, **peek** at the next, and decide:

- If the next char extends the operator → emit the longer token.
- Otherwise → emit the shorter token.

This is **maximal munch** — the classic lexer rule "always grab the longest valid token."

Operators handled this way:

Single	Double	Triple-extended
<code>!</code>	<code>!=</code>	—
<code>=</code>	<code>==</code>	<code>===</code> is detected by the parser as <code>==</code> followed by <code>=</code> (we deliberately don't make a <code>TT_TRIPLEEQ</code> token; the parser issues a friendlier error)
<code><</code>	<code><=</code>	—
<code>></code>	<code>>=</code>	—
<code>+</code>	<code>++</code> , <code>+=</code>	—
<code>-</code>	<code>--</code> , <code>-=</code>	—
<code>*</code>	<code>**</code> , <code>*=</code>	—
<code>/</code>	<code>/=</code> , also <code>//</code> (single-line comment), <code>/*</code> (multi-line comment)	—
<code>%</code>	<code>%=</code>	—
<code>&</code>	<code>&&</code>	— (single <code>&</code> is captured as <code>TT_SINGLE_AND</code> so the parser can flag it as invalid)
<code>\</code>	<code>`</code>	<code>\</code>

Why we emit `TT_SINGLE_AND` and `TT_SINGLE_OR`: if the user writes `a & b`, the lexer doesn't crash — it produces a token with type `&`. The parser's grammar has no rule that accepts `&` alone, so it produces the error *"Invalid operator '&'. Did you mean '&&'?"*. By passing it through as a token, error reporting happens at the layer that knows the most context.

Defense answer for maximal munch: *"For multi-character operators we use the maximal-munch rule. We read one character, peek the next, and if together they form a longer operator we emit the longer token. This is what a textbook lexer does and resolves all ambiguity between e.g. `=` (assignment) and `==` (equality)."*

Defense answer for `&` and `|`: *"We tokenize single ampersand and single pipe as `TT_SINGLE_AND` and `TT_SINGLE_OR` — token types that the parser explicitly does not accept. This means the user gets a syntax error at the parser layer with the full grammar context, instead of a misleading lexical error."*

5.4 Newline, tab, space — whitespace handling (lines 1286-1313)

```
elif self.current_char == '\n':
    pos = self.pos.copy()
    if tokens and tokens[-1].type != TT_NL:
        tokens.append(Token(TT_NL, "\\n", line, pos.col))
    while self.current_char == '\t' or self.current_char == ' ' or self.current_char == '\n':
        if self.current_char == '\t' or self.current_char == ' ':
            self.advance()
        else:
            line += 1
            self.advance()
    continue

elif self.current_char == '\t':
    ident_str = self.current_char
    pos = self.pos.copy()
    while self.current_char == '\t':
        self.advance()
    continue

elif self.current_char == ' ':
    ident_str = self.current_char
    pos = self.pos.copy()
    self.advance()
    while self.current_char == ' ':
        self.advance()
    continue
```

What this block does:

- A single `\n` token is emitted to mark the end of a logical line, but **only if** the last token wasn't already a `\n` — this collapses multiple consecutive blank lines into one newline token.
- The inner `while` loop advances past *all* contiguous whitespace (spaces, tabs, newlines), so the next iteration of the outer loop starts at a real character.
- For tabs and spaces alone, the consumed whitespace is discarded entirely (no token emitted).

Why emit `TT_NL` tokens at all when the parser is configured to skip them? Because the lexer's *line counter* depends on counting newlines. The token is also useful for error recovery and for the IDE's lexeme view. The parser is configured with `skip_token_types={'\n'}` so it transparently ignores them during parsing.

Why the `if tokens and tokens[-1].type != TT_NL` guard: Prevents a sequence like `\n\n\n\n` from emitting four `TT_NL` tokens — only one is needed.

Defense answer: "Whitespace is consumed but not emitted as tokens, except newlines. Newlines are emitted exactly once per logical line break so the parser's line tracking and the IDE's lexeme view stay accurate. The parser is configured to skip newline tokens during shift operations."

5.5 Forward slash — division, comments, and divide-assign (lines 1318-1370)

```
elif self.current_char == "/":
    ident_str = self.current_char
    pos = self.pos.copy()
```

```

self.advance()

if self.current_char == "/": # Single-line comment: // comment text
    ident_str += self.current_char
    self.advance()
    while self.current_char is not None and self.current_char != "\n":
        ident_str += self.current_char
        self.advance()
    continue

elif self.current_char == "*": # Multi-line comment: /* ... */
    ident_str += self.current_char
    self.advance()
    found_close = False
    while self.current_char is not None:
        if self.current_char == "*" and self.pos.index + 1 < len(self.source_code) and self.source_code[
            self.pos.index + 1] == "/":
            ident_str += "*/"
            self.advance()
            self.advance()
            found_close = True
            break
        else:
            ident_str += self.current_char
            if self.current_char == "\n":
                line += 1
            self.advance()
    if not found_close:
        errors.append(LexicalError(pos, f"Missing closing '*/' after '{ident_str}'"))
        continue
    continue

elif self.current_char == "=":
    ident_str += self.current_char
    self.advance()
    tokens.append(Token(TT_DIVEQ, ident_str, line, pos.col))
    continue

else:
    ...
    tokens.append(Token(TT_DIV, ident_str, line, pos.col))
    continue

```

What this block does: the `/` character is a four-way fork:

1. `//` — single-line comment. Consume everything up to (but not including) the next newline. **Comments are NOT emitted as tokens** — they vanish from the token stream entirely.
2. `/*` — multi-line comment. Consume characters until `*/` is found, tracking line numbers internally. If EOF is reached before `*/`, report *"Missing closing '/*'"*.
3. `/=` — divide-assign operator.
4. Plain `/` — division operator.

Why comments don't become tokens: they have zero meaning to the parser or any later stage. Suppressing them at the lexer level keeps every later layer simpler.

Multi-line comment edge case — line counting: because a `/* ... */` block can span many lines, we explicitly increment `line` whenever we consume a newline inside the comment. Otherwise error messages after a multi-line comment would point at the wrong line.

Defense answer: *"The forward slash has four meanings — comment, multi-line comment, divide-assign, division. We resolve them by peeking at the next character. Comments are consumed but never emitted; they don't reach the parser. Unclosed multi-line comments produce a lexical error with the position of the opening `/*`."*

5.6 Number scanner — integers and doubles (lines 1418-1625)

```
elif self.current_char in ZERODIGIT:
    dot_count = 0
    ident_str = ""
    pos = self.pos.copy()
    integer_digit_count = 0
    fractional_digit_count = 0

    # Read all digits before decimal point
    while self.current_char is not None and self.current_char in ZERODIGIT:
        integer_digit_count += 1
        ident_str += self.current_char
        self.advance()

    # Check for decimal point (converts to double/float)
    if self.current_char == ".":
        # ... read fractional part ...
        # ... validate digit limits ...
        tokens.append(Token(TT_DOUBLELIT, ident_str, line, pos.col))
        continue

    # No decimal point — emit as integer
    if integer_digit_count > 8:
        # Process integer part in chunks of 8: each chunk beyond the valid last ≤8 digits is an error
        ...
    tokens.append(Token(TT_INTEGERLIT, remaining, line, pos.col))
    continue
```

What this block does: When the current char is a digit (0-9):

1. Read all consecutive digits — the integer part.
2. If the next character is `.`, this is a double — read the fractional part too.
3. Validate against GAL's digit limits:
 - **Integer** (seed): max 8 digits.
 - **Double** (tree): max 15 digits before decimal, max 8 digits after.
1. Format the value (strip leading zeros from integer part, drop trailing zeros from fractional part, ensure at least one fractional digit).
2. Emit `TT_INTEGERLIT` or `TT_DOUBLELIT`.

Edge case — leading zero stripping:

```
remaining = remaining.lstrip("0") or "0"
```

The `or "0"` is a Python idiom: if `lstrip("0")` returns the empty string (i.e., the input was all zeros like `"0000"`), keep one zero so the number isn't lost.

Edge case — trailing zero handling on doubles:

```
fractional_part = (parts[1] if len(parts) > 1 else "").rstrip("0") or "0"
if fractional_part == "0":
    num_str = f"{integer_part}.0"
else:
    num_str = f"{integer_part}.{fractional_part}"
```

3.140 becomes 3.14; 3.000 becomes 3.0 (we always keep at least one zero after the decimal so it's still recognizable as a double). This keeps display consistent.

Defense answer: *"The number scanner reads all consecutive digits, then checks for a decimal point. We enforce the digit limits documented in the GAL specification — 8 digits for integers, 15 left and 8 right for doubles. We also normalize the displayed value: strip leading zeros, drop trailing zeros after the decimal but keep at least one."*

5.7 String literal scanner — "... " (lines 1631-1692)

```
elif self.current_char == '"':
    string = ''
    pos = self.pos.copy()
    escape_character = False
    string += self.current_char # opening quote
    self.advance()

    escape_characters = {
        'n': '\n', 't': '\t',
        '{': '\\{', '}': '\\}',
        '"': '\\"', '\\': '\\\\',
    }

    has_string_error = False
    while self.current_char is not None and (self.current_char != '"' or escape_character):
        if escape_character:
            if self.current_char in escape_characters:
                string += escape_characters[self.current_char]
            else:
                errors.append(LexicalError(pos, f"Invalid escape sequence '\\{self.current_char}'..."))
                has_string_error = True
            escape_character = False
        else:
            if self.current_char == '\\':
                escape_character = True
            elif self.current_char == '\n':
                break # newline ends the string (unclosed)
            else:
                string += self.current_char
        self.advance()

    if self.current_char == '"':
        string += self.current_char
        self.advance()
    else:
        errors.append(LexicalError(pos, f"Missing closing '\"' for string literal"))
        continue

    tokens.append(Token(TT_STRINGLIT, string, line, pos.col))
    continue
```

What this block does:

1. Consume the opening `"`.
2. Read characters until the closing `"`. The `escape_character` flag tracks whether the next character is an escape continuation.
3. Recognized escapes: `\n`, `\t`, `\{`, `\}`, `\"`, `\\`. Anything else triggers an *"Invalid escape sequence"* error.
4. A literal newline (not preceded by `\`) inside a string ends the string scan; if the next char is not `"`, the string is unclosed.
5. Emit `TT_STRINGLIT` whose value is the **fully-parsed text** including the quotes (the parser/interpreter peel the quotes off later).

Why escape sequences are processed at the lexer: the value stored in the token is the **real** string (with `\n` already converted to a newline char). This matches how C compilers do it — the lexer is the single point that handles escapes, so every later stage sees the actual text the user intended.

Defense answer: *"String literals are scanned with a small state machine that tracks whether the next character is part of an escape sequence. Recognized escapes are `\\n`, `\\t`, `\\{`, `\\}`, `\\\"`, `\\\\`. Unclosed strings produce a lexical error with the position of the opening quote so the IDE can highlight the right place."*

5.8 Character literal scanner — `'a'` (lines 1698-1777)

```
elif self.current_char == '"':
    string = ''
    char = ''
    pos = self.pos.copy()
    string += self.current_char
    self.advance()
    has_error = False

    while self.current_char is not None and self.current_char != '"':
        if self.current_char == '\n':
            break
        elif self.current_char == '\\':
            string += self.current_char
            self.advance()
            if self.current_char in '"\\nt':
                char += f"\\{self.current_char}"
                string += self.current_char
            else:
                errors.append(LexicalError(pos, f"Invalid escape sequence..."))
                has_error = True
                break
        else:
            string += self.current_char
            char += self.current_char
            self.advance()

    if self.current_char == '"':
        string += self.current_char
        self.advance()
    else:
        errors.append(LexicalError(pos, f"Missing closing quote..."))
        continue

    inner = char.strip()
```

```

if len(inner) == 0:
    # Empty char literal '' defaults to a space character
    ...
elif len(inner) > 1:
    errors.append(LexicalError(pos, f"Character literal must contain exactly one character..."))
    continue

tokens.append(Token(TT_CHARLIT, string, line, pos.col))
continue

```

What this block does: Like strings, but enforces **exactly one character** between the quotes. Multi-char content like `'AB'` is rejected with *"Character literal must contain exactly one character"*.

Special cases:

- Empty `''` is treated as a space character (defensive default).
- Escape sequences `\n`, `\t`, `\\`, `\'` are accepted and treated as one character.
- Newline inside the literal ends scanning; reported as unclosed.

Defense answer: *"A character literal is exactly one character (or one escape sequence) inside single quotes. We share the escape logic with strings but require length 1. Empty `''` defaults to space because some legacy programs rely on this."*

5.9 Backtick — concatenation operator (lines 1779-1789)

```

elif self.current_char == '`':
    pos = self.pos.copy()
    ident_str = self.current_char
    self.advance()
    if self.current_char is not None and self.current_char not in set(ALPHANUM + '(!"\' ' + '\t' + '\n'):
        errors.append(LexicalError(pos, f"Invalid delimiter '{self.current_char}' after '{ident_str}'"))
        self.advance()
        continue
    tokens.append(Token(TT_CONCAT, ident_str, line, pos.col))
    continue

```

What this block does: GAL uses backtick ``` as the ****string-concatenation**** operator (where most languages would use `+` or `..`). It's a single-character token. The delimiter check ensures the next character is something that could legally start an operand (an identifier, a literal, a paren, etc.).

Defense answer: *"GAL uses backtick as the concatenation operator. We chose it because the `+` symbol is reserved for arithmetic and using a distinct character makes string-vs-number context obvious to the reader."*

5.10 The catch-all `else` — illegal characters (lines 1791-1826)

```

else:
    pos = self.pos.copy()
    char = self.current_char

    if char == '_':
        # Underscore at start of identifier – special error
        ...
        if temp_str in reserved_words:

```

```

        errors.append(LexicalError(pos, f"Reserved word cannot start with a symbol: '{temp_str}'))
    else:
        errors.append(LexicalError(pos, f"Identifiers cannot start with a symbol: '_{...}'"))
    ...
    continue
else:
    self.advance()
    errors.append(LexicalError(pos, f"Illegal Character '{char}'))
    continue

```

What this block does: if no branch above matched, the character is illegal. We emit a lexical error and skip past the bad character so scanning can continue.

Special case for `_`: GAL identifiers must start with a letter, not an underscore. If the user wrote `_water`, we want to give a more helpful error than "illegal character" — we peek ahead, check if the rest of the word is a reserved word, and emit a clearer message: *"Reserved word cannot start with a symbol: `_water`"*.

Defense answer: *"The default branch reports any character we don't recognize as an illegal-character lexical error. We have a special path for leading underscore because that's a common typo where the user expected an identifier; we tell them specifically that identifiers must start with a letter."*

5.11 EOF handling and return (lines 1828-1832)

```

if self.current_char is None:
    tokens.append(Token(TT_EOF, "", line, pos.col))

return tokens, errors

```

What this block does: after the main `while` loop ends (EOF reached), append a synthetic `TT_EOF` token. This is critical — the parser's LL(1) table uses `EOF` as the end-marker terminal. Without this token, the parser would not know when to stop.

Defense answer: *"The lexer always appends an EOF token at the end of input. The parser is configured with `end_marker='EOF'` and uses this synthetic token to know when valid input has ended."*



6. DEFENSE QUESTION PREPARATION

Q: Why is the keyword scanner hand-written instead of using regex or a dictionary?

"Three reasons. First, it directly mirrors the transition diagrams in our compiler proposal — each `if` chain is one DFA path, so a panel can trace the implementation back to the spec. Second, GAL has per-keyword delimiter rules (e.g., `cultivate` requires `:` or `(` to follow) — a regex approach would need a separate validator. Third, the hand-written FSM cleanly falls through to the generic identifier loop when a keyword prefix doesn't complete — `seedling` is recognized as one identifier, not as `seed` + `ling`."

Q: How does the lexer distinguish `seed` (keyword) from `seedy` (identifier)?

"After spelling out the keyword `seed`, the next character must be a valid delimiter — whitespace, `;`, `(`, etc. If the next character is alphanumeric (y in `seedy`), the FSM exits and control falls through to the generic identifier loop, which builds an `id` token whose value is `seedy`. So `seedy` is one identifier token, not two."

Q: Why does `~` produce two different kinds of tokens?

"When `~` is followed by a digit, we fold the sign into the literal at lex time — the result is a single `intlit` or `dbllit` token whose value carries the `~`. When `~` is followed by an identifier or paren, we emit a separate `~` operator token. This means the parser doesn't need a unary-minus production for literals — they already arrive negated. For variables, the parser uses the `~` token as a unary prefix in expressions."

Q: Why are comments stripped at the lexer instead of just turned into tokens that the parser ignores?

"Comments have no semantic role anywhere in the compiler. Carrying them through as tokens would complicate every later stage. Stripping at lex time is cleanest. We do still track line numbers carefully through multi-line comments so error messages stay accurate."

Q: How does the lexer recover from errors? Does it stop on the first one?

"It never stops. Every error path appends a `LexicalError` to the errors list, calls `self.advance()` to skip past the bad character, and continues the loop. This way the user sees all lexical errors in one pass instead of fixing them one at a time."

Q: How does the lexer handle Windows line endings?

"In the constructor we strip `\\r` from the source: `source_code.replace('\\r', '')`. After that, the rest of the lexer only ever has to consider `\\n` for line endings. This is a defensive normalization that prevents Windows users from seeing wrong line counts."

Q: What is maximal munch, and where do you apply it?

"Maximal munch is the lexer rule 'always read the longest valid token.' We apply it to every multi-character operator: `=` vs `==`, `<` vs `<=`, `+` vs `++` vs `+=`, etc. Each branch peeks at the next character and emits the longer token if applicable, otherwise the shorter one. There are no ambiguous cases that survive."

Q: What happens if a string literal is unclosed at end of file?

"We detect this in the main while loop: `while self.current_char is not None and (self.current_char != '\"' or escape_character)`. If we exit the loop and `self.current_char` is `None` (EOF) or `\\n` (newline), we emit `LexicalError(pos, 'Missing closing \". . .')` with the position of the opening quote. The lexer then returns. The parser doesn't run because the server short-circuits on lexical errors."

Q: Why does the lexer enforce digit limits like 8 digits for integers?

"These are documented constraints in the GAL specification. An 8-digit cap means the user cannot accidentally write `9999999999999999` and overflow the runtime's 32-bit integer logic. The lexer enforces them up front so the user gets a clear error pointing at the bad number, rather than a confusing

arithmetic-overflow at runtime."

Q: Why are there two functions — run and lex?

"run is a legacy entry point kept for backward compatibility with grading scripts. lex is the production entry used by server.py — it does the same scanning but additionally converts each LexicalError object to its formatted string via as_string(), because the JSON layer needs strings, not objects."



7. SIMPLE WALKTHROUGH EXAMPLE

Sample source:

```
root() {  
    seed age = 10;  
    plant(age);  
    reclaim;  
}
```

How the lexer scans this character by character:

Step	pos	current_char	Branch entered	Output
1	(0,1,0)	r	current_char in ALPHA → r branch → spell root	Token('root', 'root', 1, 0)
2	(4,1,4)	(	current_char == "("	Token('(', '(', 1, 4)
3	(5,1,5)	)	current_char == ")"	Token(')', ')', 1, 5)
4	(6,1,6)		current_char == ' ' → consume whitespace	(no token)
5	(7,1,7)	{	current_char == "{"	Token('{', '{', 1, 7)
6	(8,1,8)	\n	current_char == '\n'	Token('\n', '\n', 1, 8)
7	(9,2,0)		space-skip loop	(no token)
8	(13,2,4)	s	s branch → spell seed	Token('seed', 'seed', 2, 4)
9	(17,2,8)		whitespace	(no token)

Step	pos	current_char	Branch entered	Output
10	(18,2,9)	a	a is not a keyword start letter → identifier loop	Token('id', 'age', 2, 9)
11	(21,2,12)		whitespace	(no token)
12	(22,2,13)	=	= branch → next char is space, not = → assignment	Token('=', '=', 2, 13)
13	(23,2,14)		whitespace	(no token)
14	(24,2,15)	1	digit branch → read 10	Token('intlit', '10', 2, 15)
15	(26,2,17)	;	; branch	Token(';', ';', 2, 17)
16	(27,2,18)	\n	newline	Token('\n', '\n', 2, 18)
17	...	...	continues for plant(age);, reclaim; }	...
last	EOF	None	loop ends	Token('EOF', '', 5, 1)

The final (tokens, errors) returned to the caller:

- tokens is the list above (22 tokens including 4 newlines and EOF).
- errors is [] because the source is well-formed.

If instead the user typed seed age = 10@; (with an illegal @):

Step	current_char	Action
...	1 then 0	digit loop builds 10
	@	next char isn't ., isn't whitespace, isn't a valid delimiter for an integer; the digit branch's delimiter check at line 1418's exit triggers → LexicalError(pos, "Invalid delimiter '@' after '10'"). Lexer advances past @ and continues.
	;	normal ; token emitted

The result is the same token list as the valid case, but `errors` contains one entry. The server sees `errors` is non-empty and short-circuits the pipeline at the lexical stage.



8. DEFENSE-READY EXPLANATION (memorize this)

"lexer.py is the scanner of my compiler. It receives GAL source code as a Python string. It walks the source character by character in one pass, dispatching each character to the correct scanner branch — keywords, identifiers, numbers, strings, characters, operators, comments, or whitespace. It produces a list of Token objects with type, value, line, and column, plus a list of LexicalError objects for any problems it finds. It never raises an exception — every error is collected and the scanner advances past the bad character so we can report multiple errors in one pass. Keywords are recognized via a hand-written transition diagram that mirrors the FSMs in our compiler proposal. The ~ character is special: when it precedes a digit we fold it into a negative literal at scan time; when it precedes an identifier we emit it as a separate negation operator. Multi-character operators use the maximal-munch rule — we always emit the longest valid token. Comments are consumed but not emitted; they vanish from the token stream entirely. Whitespace is skipped except for newlines, which we emit once per logical line break so the parser can track lines. The lexer always appends a synthetic EOF token at the end so the parser knows when input ends. It exposes two functions to the rest of the system — run (legacy, returns raw error objects) and Lex (production, returns string-formatted errors for JSON serialization)."



Next file in the defense-prep series: `cfg.py` and `Gal_Parser.py` — the LL(1) grammar and the table-driven parser.